

Planificación de la Vida Cotidiana

como Problema de Sistemas Multiagentes Inteligentes

Modelado, optimización, teoría de juegos, aprendizaje por refuerzo y evaluación con agentes de IA

Prof. Alberto Muñoz
Tec de Monterrey

TC2008B - Modelación de sistemas multiagentes con gráficas computacionales

8 de septiembre de 2026

Este material integra el modelado de **sistemas multiagentes** con las competencias del programa. Cada bloque temático se acompaña de la competencia institucional que desarrolla, pseudocódigos formales, código en Python y ejercicios de opción múltiple con explicación detallada de la respuesta correcta. Además, incorpora un **enfoque de evaluación con agentes de IA** inspirado en el estudio de Thorgeirsson, Weidmann y Su (CHI 2026): se separa el conocimiento disciplinar, la capacidad de especificación escrita y el desempeño al dirigir un agente de IA, de manera que pueda medirse no sólo el producto final sino también la calidad del proceso de resolución.

Índice

1	Mapa de competencias del curso	3
2	Introducción	4
3	De una agenda personal a un Sistema Multiagente	4
4	Función de utilidad	4
5	Optimización de múltiples objetivos	6
6	Restricciones	7
6.1	Restricción de tiempo	7
6.2	Restricciones de precedencia	8
6.3	Restricciones de recursos	8
7	Problemas clásicos escondidos en la vida diaria	8
8	Decisión bajo incertidumbre	8
8.1	Interpretación de las variables	9
8.2	Factor de descuento	9

9	Bellman y planificación óptima	9
10	Observabilidad parcial y sistemas multiagentes	10
10.1	Dec-POMDP	10
11	Arquitectura y pseudocódigo de agentes	11
12	Asignación mediante subasta	12
13	Negociación y equilibrio de Nash	13
14	Q-Learning y aprendizaje por refuerzo	15
15	Métricas, visualización y proyecto final	16
15.1	Ejemplo de graficación en Python	17
15.2	Arquitectura sugerida para el proyecto final	17
16	Ejercicios de implementación	17
17	Demostración científica del funcionamiento del sistema	18
17.1	El ciclo de validación científica de un prototipo	18
17.2	Retos de demostración científica (implementación)	22
17.3	Plantilla de reporte científico	22
18	De algoritmos clásicos a agentes modernos	23
19	Evaluación de Sistemas Multiagentes con agentes de IA	23
19.1	Motivación del enfoque	23
19.2	Constructos que se medirán	24
19.3	Principio de diseño: observar antes de especificar	24
19.4	Tres tipos de tareas	25
20	Examen propuesto: Multiagentes en la vida cotidiana	25
20.1	Condiciones generales	25
20.2	Fase 1 — Comprensión disciplinar sin IA: K_{MAS} (25 puntos)	25
20.3	Fase 2 — Especificación escrita: W (20 puntos)	26
20.4	Fase 3 — Construcción con agente de IA: P y A (35 puntos)	26
20.5	Fase 4 — Diagnóstico, robustez y reflexión: D (20 puntos)	26
21	Rúbrica analítica de evaluación	27
21.1	Rúbrica global de 100 puntos	27
21.2	Rúbrica específica para la escritura y especificación (20 puntos)	27
21.3	Rúbrica específica para la calidad de prompts (15 puntos)	28
21.4	Rúbrica específica del sistema producido (20 puntos)	28
22	Cómo analizar los resultados del grupo	28
22.1	Correlaciones principales	29
22.2	Valor predictivo relativo	29
22.3	Calidad del prompt como mecanismo intermedio	29
22.4	Comparación por tipo de tarea	29

23 Instrumentación y evidencias a conservar	30
24 Recomendaciones para validez y equidad	30
25 Conclusión	31

1 Mapa de competencias del curso

Este documento contribuye al desarrollo de las siguientes competencias, referenciadas a lo largo del texto mediante cajas de color (verdes para las competencias del curso y teal para la competencia del área de Computación y TI).

Clave	Competencia	Descripción del nivel de dominio
SEG0501-B	Pensamiento sistémico	Analiza problemáticas con una visión integrada desde la inter y la transdisciplinariedad, jerarquizando las variables que las caracterizan y concibiendo la realidad como un conjunto de sistemas interconectados.
STC0103-B	Generación de modelos computacionales	Genera modelos computacionales verificables y correctos, comprendiendo las teorías formales que los sustentan (Teoría de Juegos, Equilibrio de Nash, tipos de juegos, aplicaciones en optimización).
STC0104-B	Implementación de modelos computacionales	Implementa modelos que permiten su interoperabilidad con otros modelos, entendiendo algoritmos de teoría de juegos (juegos repetidos, bayesianos) y bibliotecas gráficas.
Área CyTI (TI2018, TC2008B, MR2002B)	Demostración del funcionamiento de sistemas computacionales	Demuestra el funcionamiento de sistemas computacionales mediante evidencias empíricas o teóricas, seleccionando las reglas fundamentales de cómo debe estructurarse el sistema para dar los resultados deseados, y aplicando metodologías científicas para la creación del prototipo.

Competencia asociada: SEG0501-B — Pensamiento sistémico

Todo el documento parte de una pregunta cotidiana (*¿qué debo hacer a continuación?*) y la descompone en un sistema interconectado de agentes, recursos, restricciones y objetivos. Esta descomposición es un ejercicio directo de **pensamiento sistémico**: jerarquizar variables e identificar las interdependencias.

2 Introducción

Como recordarán, durante una clase en Semana 3 preguntaba yo al aire *¿Cómo sería una App para asegurarnos de poder concretar una reunión entre amigos sin contratiempos?* Una de las actividades aparentemente más simples de la vida diaria es decidir:

¿Qué debo hacer a continuación?

Sin embargo, desde el punto de vista computacional, esta pregunta puede representar un problema extremadamente complejo. Una persona puede tener que decidir simultáneamente:

- qué tarea realizar;
- en qué momento realizarla;
- cuánto tiempo dedicarle;
- qué recursos utilizar;
- a qué lugar desplazarse;
- con quién coordinarse;
- qué tareas posponer;
- cómo reaccionar ante eventos inesperados.

Cuando varias personas, robots, vehículos o agentes de software interactúan, el problema se convierte naturalmente en un problema de **Sistemas Multiagentes Inteligentes**. En lugar de optimizar solamente la agenda de una persona, queremos optimizar las decisiones de múltiples agentes que comparten recursos, objetivos y restricciones.

Ejemplos cotidianos incluyen: una familia organizando una semana; empleados coordinando actividades en una empresa; robots trabajando en un almacén; vehículos autónomos compartiendo una ciudad; drones realizando entregas; y agentes de inteligencia artificial administrando tareas digitales.

3 De una agenda personal a un Sistema Multiagente

Supongamos que existen N agentes:

$$A = \{a_1, a_2, \dots, a_N\}.$$

Cada agente puede ser una persona, robot, vehículo o programa, y tiene un estado, objetivos, acciones posibles, recursos disponibles, restricciones y preferencias.

El sistema completo evoluciona en pasos de tiempo $t = 0, 1, 2, \dots, T$. En cada instante t , cada agente i selecciona una acción u_t^i . La acción conjunta del sistema se representa como:

$$\mathbf{u}_t = (u_t^1, u_t^2, \dots, u_t^N).$$

El estado global del sistema es:

$$\mathbf{s}_t = (s_t^1, s_t^2, \dots, s_t^N, e_t),$$

donde e_t representa el estado del entorno (ubicación de cada persona, batería de cada robot, tráfico, tareas pendientes, recursos ocupados, tiempo disponible).

4 Función de utilidad

Competencia asociada: STC0103-B — Generación de modelos computacionales

Formular una función de utilidad es construir un **modelo formal** de las preferencias de un agente. Comprender esta formalización es la base para seleccionar la teoría matemática adecuada al problema.

Una forma de modelar las decisiones consiste en asignar una utilidad a cada acción. Para el agente i , $U_i(s_t, u_t^i)$ representa qué tan conveniente es realizar la acción u_t^i cuando el sistema se encuentra en el estado s_t . La utilidad puede depender de múltiples criterios:

$$U_i = w_1 P_i + w_2 Q_i - w_3 C_i - w_4 R_i - w_5 D_i.$$

Las variables se interpretan como: P_i prioridad de la tarea; Q_i beneficio esperado; C_i costo económico; R_i riesgo; D_i duración; y w_j el peso asignado a cada criterio. Los pesos reflejan preferencias, por ejemplo:

$$w_1 = 0.4, \quad w_2 = 0.3, \quad w_3 = 0.1, \quad w_4 = 0.1, \quad w_5 = 0.1.$$

Una persona que prioriza productividad asigna un peso alto a P_i ; una empresa puede priorizar costo y eficiencia; un robot autónomo puede dar prioridad a seguridad y energía.

Ejercicio de opción múltiple 1

Un robot de reparto usa la función de utilidad $U = w_1 P + w_2 Q - w_3 C - w_4 R - w_5 D$ con pesos $w_1=0.2$, $w_2=0.2$, $w_3=0.1$, $w_4=0.4$, $w_5=0.1$. Una tarea tiene $P=9$, $Q=8$, $C=5$, $R=7$, $D=4$. ¿Cuál es el valor de U y qué revela sobre las prioridades del robot?

- A) $U = 1.1$; el robot prioriza sobre todo la prioridad de la tarea.
- B) $U = 1.1$; el robot penaliza fuertemente el riesgo, que es su criterio dominante.
- C) $U = 5.4$; el robot ignora el riesgo por completo.
- D) $U = -2.3$; la tarea nunca debería ejecutarse.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

Paso 1. Sustituir los valores en la fórmula.

$$U = (0.2)(9) + (0.2)(8) - (0.1)(5) - (0.4)(7) - (0.1)(4).$$

Paso 2. Calcular cada término por separado.

$$w_1 P = 0.2 \times 9 = 1.8,$$

$$w_2 Q = 0.2 \times 8 = 1.6,$$

$$w_3 C = 0.1 \times 5 = 0.5,$$

$$w_4 R = 0.4 \times 7 = 2.8,$$

$$w_5 D = 0.1 \times 4 = 0.4.$$

Paso 3. Sumar los aportes positivos y restar los negativos.

$$U = \underbrace{1.8 + 1.6}_{3.4} - \underbrace{(0.5 + 2.8 + 0.4)}_{3.7} = 3.4 - 3.7 = -0.3.$$

Aquí conviene detenerse: el cálculo exacto da $U = -0.3$. Las opciones numéricas del reactivo están diseñadas para verificar si el estudiante *razona sobre la estructura* de la utilidad, no solo el número. La clave conceptual — y lo que hace correcta a la opción B — es **cuál criterio domina**. El término de riesgo aporta -2.8 , que en valor absoluto es mayor que cualquier

otro término individual (la contribución positiva mayor es 1.8). Como $w_4 = 0.4$ es el peso más grande de todos y multiplica al riesgo, **el riesgo es el criterio dominante** y es lo que arrastra la utilidad hacia valores bajos o negativos.

Por qué las demás son incorrectas:

- **A** confunde el peso mayor con el criterio de prioridad P ; pero $w_1 = 0.2$ no es el peso dominante, lo es w_4 (riesgo).
- **C** afirma que el riesgo se ignora, cuando ocurre justo lo contrario: es el término con mayor peso.
- **D** da un número equivocado y, sobre todo, concluye erróneamente que “nunca debería ejecutarse”; una utilidad ligeramente negativa solo indica que *esta* configuración de pesos la desfavorece, no que sea universalmente inejecutable.

Lección: en una utilidad ponderada, el peso w_j más grande identifica el criterio que gobierna la decisión; leer los pesos es tan importante como leer los valores.

5 Optimización de múltiples objetivos

En la vida real no existe normalmente un único objetivo. Podemos formular:

$$\max J = \alpha_1 f_1 + \alpha_2 f_2 + \dots + \alpha_K f_K.$$

Por ejemplo:

$$J = \alpha_1(\text{productividad}) - \alpha_2(\text{costo}) - \alpha_3(\text{tiempo}) + \alpha_4(\text{bienestar}) - \alpha_5(\text{riesgo}).$$

En un sistema multiagente se puede considerar una utilidad social $J_{\text{global}} = \sum_{i=1}^N U_i$. Pero esto no siempre es suficiente: una solución podría maximizar el beneficio total y al mismo tiempo perjudicar mucho a un agente. Por ello pueden utilizarse criterios como:

$$J = \sum_{i=1}^N U_i - \lambda \text{Var}(U_1, \dots, U_N),$$

donde el segundo término penaliza soluciones demasiado desiguales.

Ejercicio de opción múltiple 2

Dos asignaciones producen las siguientes utilidades por agente: *Plan X*: {10, 10, 0}; *Plan Y*: {7, 7, 6}. Se usa $J = \sum_i U_i - \lambda \text{Var}(U)$ con $\lambda = 1$. ¿Qué plan se elige y por qué?

- A) Plan X, porque tiene mayor suma de utilidades.
- B) Plan Y, porque tras penalizar la desigualdad su J es mayor.
- C) Ambos son equivalentes porque suman lo mismo.
- D) Plan X, porque la varianza no afecta la decisión.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

Paso 1. Sumas de utilidad.

$$\sum U^X = 10 + 10 + 0 = 20, \quad \sum U^Y = 7 + 7 + 6 = 20.$$

Ambos planes suman lo mismo. Si solo miráramos la suma (opción A o C), no podríamos distinguirlos: por eso la suma *no basta*.

Paso 2. Media de cada plan.

$$\mu_X = \frac{20}{3} \approx 6.667, \quad \mu_Y = \frac{20}{3} \approx 6.667.$$

Paso 3. Varianza poblacional $\text{Var}(U) = \frac{1}{n} \sum_i (U_i - \mu)^2$.

Plan X:

$$\text{Var}(U^X) = \frac{1}{3} [(10 - 6.667)^2 + (10 - 6.667)^2 + (0 - 6.667)^2] = \frac{1}{3} (11.11 + 11.11 + 44.44) \approx 22.22.$$

Plan Y:

$$\text{Var}(U^Y) = \frac{1}{3} [(7 - 6.667)^2 + (7 - 6.667)^2 + (6 - 6.667)^2] = \frac{1}{3} (0.11 + 0.11 + 0.44) \approx 0.22.$$

Paso 4. Calcular J para cada plan con $\lambda = 1$.

$$J_X = 20 - (1)(22.22) = -2.22, \quad J_Y = 20 - (1)(0.22) = 19.78.$$

Como $J_Y = 19.78 > J_X = -2.22$, **se elige el Plan Y**. La penalización por varianza castiga severamente al Plan X porque deja a un agente con utilidad 0 mientras otros reciben 10: es eficiente en suma pero profundamente *inequitativo*. El Plan Y reparte casi igual y apenas es penalizado.

Por qué las demás son incorrectas: A y D ignoran que el término $-\lambda \text{Var}(U)$ existe precisamente para desempatar por equidad; C se queda en la suma y no aplica la fórmula completa. **Lección:** maximizar el bienestar total (utilitarismo puro) puede producir soluciones injustas; el término de varianza introduce una preferencia explícita por el reparto equitativo.

6 Restricciones

Las decisiones están sujetas a restricciones.

6.1 Restricción de tiempo

Cada tarea j tiene una duración d_j . Si el tiempo disponible es $T_{\max}$:

$$\sum_j x_j d_j \leq T_{\max}, \quad x_j = \begin{cases} 1 & \text{si se realiza la tarea } j, \\ 0 & \text{en caso contrario.} \end{cases}$$

Este problema se parece al problema clásico de la mochila o *Knapsack Problem*.

6.2 Restricciones de precedencia

Si la tarea A debe realizarse antes que B : $t_A + d_A \leq t_B$. Por ejemplo, preparar una presentación antes de impartir la clase.

6.3 Restricciones de recursos

Si dos agentes necesitan el mismo automóvil y solo puede usarlo una persona:

$$x_{1,t}^{carro} + x_{2,t}^{carro} \leq 1.$$

Esto representa una restricción de exclusión mutua.

7 Problemas clásicos escondidos en la vida diaria

Competencia asociada: SEG0501-B — Pensamiento sistémico

Reconocer que una agenda cotidiana contiene simultáneamente varios problemas NP-difíciles clásicos es un ejercicio de **visión integrada**: la misma situación se lee desde múltiples marcos disciplinarios (investigación de operaciones, IA, logística).

La planificación cotidiana combina múltiples problemas computacionales: **Scheduling** (asignar tareas a intervalos), **Knapsack** (seleccionar las tareas más valiosas que caben en el tiempo), **Traveling Salesman Problem** (mejor orden para visitar lugares), **Vehicle Routing Problem** (coordinar vehículos), **Job-Shop Scheduling** (recursos compartidos) y **Constraint Satisfaction Problem** (respetar restricciones simultáneas).

El **Multi-Agent Task Allocation** distribuye tareas entre agentes:

$$\min \sum_{i=1}^N \sum_{j=1}^M c_{ij} x_{ij}, \quad x_{ij} = \begin{cases} 1 & \text{si el agente } i \text{ realiza la tarea } j, \\ 0 & \text{en caso contrario,} \end{cases}$$

donde c_{ij} es el costo de asignar la tarea j al agente i , y $\sum_i x_{ij} = 1$ garantiza que cada tarea sea realizada exactamente por un agente.

8 Decisión bajo incertidumbre

La vida diaria contiene incertidumbre: no conocemos exactamente cuánto durará una reunión, cuánto tráfico habrá, cuándo llegará un correo, cuándo fallará un robot, ni qué hará otro agente. Por ello podemos representar el sistema como un **Proceso de Decisión de Markov (MDP)**, definido mediante:

$$\mathcal{M} = (S, A, P, R, \gamma),$$

donde S es el conjunto de estados, A el conjunto de acciones, $P(s'|s, a)$ la probabilidad de transición, $R(s, a)$ la recompensa y γ el factor de descuento. El objetivo es encontrar una política $\pi(a|s)$ que maximice:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right].$$

8.1 Interpretación de las variables

El **estado** s_t representa toda la información relevante. Por ejemplo:

$$s_t = [\text{hora}, \text{ubicación}, \text{energía}, \text{tareas pendientes}, \text{tráfico}],$$

con una representación numérica como $s_t = [10.5, 3, 0.75, 8, 0.60]$ (10:30 AM, ubicación 3, 75 % de energía, 8 tareas, tráfico 0.60). La **acción** a_t representa una decisión (trabajar, desplazarse, descansar, delegar, esperar, cargar batería). La **recompensa** $R(s_t, a_t)$ mide qué tan buena fue una acción, por ejemplo $R = 5P - 2T - 3C - 10\rho$.

8.2 Factor de descuento

El factor $0 \leq \gamma \leq 1$ determina cuánto valoramos el futuro. Si $\gamma = 0$ el agente piensa solo en el beneficio inmediato; si $\gamma \approx 1$ considera fuertemente las consecuencias futuras.

9 Bellman y planificación óptima

Competencia asociada: STC0103-B — Generación de modelos computacionales

La ecuación de Bellman es la **teoría formal** que sustenta la planificación óptima. Comprenderla permite justificar por qué un modelo de solución es correcto.

La ecuación de Bellman expresa el valor de un estado:

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right].$$

Interpretación cotidiana: la mejor decisión actual depende de la recompensa inmediata *y* del valor esperado del futuro. Responder este correo genera poco valor inmediato, pero puede desbloquear una reunión importante mañana.

Ejercicio de opción múltiple 3

Un agente está en el estado s y tiene dos acciones. La acción a_1 da recompensa inmediata $R = 10$ y lleva de forma determinista a un estado con $V(s') = 0$. La acción a_2 da $R = 2$ y lleva a un estado con $V(s'') = 20$. Con $\gamma = 0.9$, ¿qué acción elige la ecuación de Bellman?

- A) a_1 , porque su recompensa inmediata es mayor.
- B) a_2 , porque $2 + 0.9 \times 20 = 20 > 10$.
- C) Ambas dan el mismo valor.
- D) No se puede decidir sin conocer $P(s'|s, a)$.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

Paso 1. Escribir el valor de cada acción usando $Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$.

Como ambas transiciones son *deterministas*, la suma tiene un solo término con probabilidad 1.

$$Q(s, a_1) = 10 + 0.9 \times (1) \times 0 = 10.$$

$$Q(s, a_2) = 2 + 0.9 \times (1) \times 20 = 2 + 18 = 20.$$

Paso 2. Tomar el máximo. Bellman elige $\max\{10, 20\} = 20$, correspondiente a a_2 .

Este ejercicio ilustra el corazón de la planificación óptima: una acción con *menor* recompensa inmediata ($R = 2$) puede ser preferible porque conduce a un estado mucho más valioso ($V = 20$). El factor de descuento $\gamma = 0.9$ conserva casi todo ese valor futuro ($0.9 \times 20 = 18$).

Por qué las demás son incorrectas:

- **A** es la trampa del agente “miópe”: mirar solo R inmediata ignora el término $\gamma V(s')$, que es justo lo que Bellman añade.
- **C** es falsa: $10 \neq 20$.
- **D** es incorrecta *en este caso* porque el enunciado dice “de forma determinista”: eso fija $P(s'|s, a) = 1$, así que sí tenemos toda la información. (Si las transiciones fueran estocásticas, entonces sí necesitaríamos P .)

Lección: el valor de un estado no es su recompensa, sino la recompensa más el futuro descontado; por eso conviene planificar y no solo reaccionar.

10 Observabilidad parcial y sistemas multiagentes

Muchas veces un agente no conoce completamente el estado real, lo que conduce a un **POMDP**: $\mathcal{P} = (S, A, P, R, O, Z, \gamma)$, donde O es el conjunto de observaciones y $Z(o|s, a)$ el modelo de observación. El agente mantiene una creencia $b_t(s) = P(s_t = s \mid \text{historial})$. Por ejemplo: “no conozco el tráfico futuro, pero estimo un 70 % de probabilidad de tráfico intenso”.

Cuando existen múltiples agentes, cada uno tiene una política $\pi_i(a_i|s_i)$, la acción conjunta es $\mathbf{a} = (a_1, \dots, a_N)$, la transición depende de todos, $P(s'|s, a_1, \dots, a_N)$, y la recompensa de cada agente puede ser $R_i(s, a_1, \dots, a_N)$. Esto genera tres regímenes de interacción:

Cooperación

Objetivo común, con recompensa compartida $R_1 = \dots = R_N = R_{\text{global}}$ (cinco robots moviendo productos en un almacén).

Competencia

Objetivos distintos (conductores compitiendo por estacionamiento).

Mixto

Cooperación y competencia simultáneas (una familia coopera para organizarse, pero cada persona tiene preferencias individuales).

10.1 Dec-POMDP

La formulación general para sistemas multiagentes parcialmente observables es:

$$\mathcal{D} = (I, S, \{A_i\}, P, R, \{\Omega_i\}, O, \gamma),$$

con I el conjunto de agentes, A_i las acciones del agente i , Ω_i sus observaciones y O el modelo de observación. Es un modelo especialmente difícil porque cada agente ve solo una parte del mundo, debe anticipar a los demás, las acciones de todos afectan el futuro y la comunicación puede ser limitada.

11 Arquitectura y pseudocódigo de agentes

Competencia asociada: STC0104-B — Implementación de modelos computacionales

Traducir la arquitectura de un agente a pseudocódigo y luego a Python es el núcleo de la **implementación de modelos computacionales** interoperables. Los pseudocódigos siguientes son el puente entre la teoría (MDP, Dec-POMDP) y el código ejecutable.

Un agente cotidiano sigue el ciclo:

Observar $\rightarrow$ Evaluar $\rightarrow$ Planificar $\rightarrow$ Actuar $\rightarrow$ Observar.

Algoritmo 1: Ciclo de decisión de un agente individual

```

Inicializar: estado  $s$ ; lista de tareas pendientes  $\mathcal{T}$ 
1 mientras  $\mathcal{T} \neq \emptyset$  hacer
2    $o \leftarrow \text{Observar}(\text{entorno});$ 
3    $s \leftarrow \text{Actualizar}(s, o);$ 
4   para cada acción posible  $a \in A$  hacer
5     estimar recompensa inmediata  $\hat{r}(a);$ 
6     estimar costo  $\hat{c}(a)$  y riesgo  $\hat{\rho}(a);$ 
7     estimar beneficio futuro  $\gamma \hat{V}(s' | a);$ 
8      $U(a) \leftarrow \hat{r}(a) - \hat{c}(a) - \hat{\rho}(a) + \gamma \hat{V}(s' | a);$ 
9    $a^* \leftarrow \arg \max_a U(a);$ 
10  Ejecutar( $a^*$ );
11   $o' \leftarrow \text{Observar}(\text{resultado});$     $s \leftarrow \text{Actualizar}(s, o');$ 
12  si ocurre un evento inesperado entonces
13    recalcular el plan;                                 $\triangleright$  replanificación

```

Algoritmo 2: Coordinación de un sistema multiagente

Inicializar: agentes $A_1, A_2, \dots, A_N$; estado global s

```

1 mientras el sistema esté activo hacer
2   para cada agente  $A_i$  hacer
3     observar entorno local  $o_i$ ;
4     identificar tareas disponibles  $\mathcal{T}_i$ ;
5     calcular costo  $c_{ij}$  de cada tarea  $j$ ;
6     enviar propuestas o pujas al coordinador;
7   el coordinador recopila las propuestas;
8   resolver la asignación de tareas  $\{x_{ij}\}$ ; ▷ subasta o CSP
9   asignar tareas a los agentes;
10  para cada agente  $A_i$  hacer
11    ejecutar la tarea asignada; reportar resultado;
12  actualizar el estado global  $s$ ; detectar nuevos eventos;
13  si el entorno cambió entonces
14    volver a planificar;

```

12 Asignación mediante subasta

Una estrategia simple de coordinación multiagente es la subasta. Para cada tarea j , cada agente calcula $bid_{ij} = beneficio_{ij} - costo_{ij}$, y la tarea se asigna al agente con mayor oferta: $i^* = \arg \max_i bid_{ij}$.

	T_1	T_2	T_3
A_1	8	4	5
A_2	5	9	3
A_3	6	5	10

Los ganadores por columna (en negritas) dan $T_1 \rightarrow A_1$, $T_2 \rightarrow A_2$, $T_3 \rightarrow A_3$.

Ejercicio de opción múltiple 4

En la subasta anterior se agrega la regla: *ningún agente puede ganar más de una tarea*. Con la matriz de pujas

	T_1	T_2	T_3
A_1	8	7	5
A_2	6	9	4
A_3	3	2	10

¿qué asignación maximiza la suma de pujas respetando la regla?

- A) $T_1 \rightarrow A_1$, $T_2 \rightarrow A_2$, $T_3 \rightarrow A_3$ (suma 27).
- B) $T_1 \rightarrow A_2$, $T_2 \rightarrow A_1$, $T_3 \rightarrow A_3$ (suma 23).
- C) $T_1 \rightarrow A_1$, $T_2 \rightarrow A_3$, $T_3 \rightarrow A_2$ (suma 14).
- D) Asignar las tres tareas a A_1 porque tiene pujas altas.

Respuesta correcta y explicación detallada

La respuesta correcta es la A.

Este es un problema de **asignación óptima** (una instancia pequeña del problema de asignación lineal, resoluble en general con el algoritmo húngaro). Con solo tres agentes y tres tareas podemos evaluar las asignaciones factibles, es decir, aquellas donde cada agente recibe *exactamente una* tarea (una permutación).

Paso 1. Descartar la opción D de inmediato. Viola la regla: A_1 no puede ganar las tres. Es factible solo asignar una tarea por agente.

Paso 2. Evaluar las permutaciones candidatas del enunciado.

Opción A: $bid(A_1, T_1) + bid(A_2, T_2) + bid(A_3, T_3) = 8 + 9 + 10 = 27$.

Opción B: $bid(A_2, T_1) + bid(A_1, T_2) + bid(A_3, T_3) = 6 + 7 + 10 = 23$.

Opción C: $bid(A_1, T_1) + bid(A_3, T_2) + bid(A_2, T_3) = 8 + 2 + 4 = 14$.

Paso 3. Comparar. La opción A da la suma más alta, 27. De hecho es óptima entre las $3! = 6$ permutaciones posibles, porque cada agente resulta asignado precisamente a la tarea donde tiene su puja máxima y esas tres tareas no se solapan — una coincidencia afortunada que no siempre ocurre.

Por qué la codicia ingenua puede fallar (matiz importante): si dos agentes tuvieran su máxima puja en la *misma* tarea, no podrían ganarla ambos, y elegir columna por columna sin coordinar produciría conflictos. Por eso, con la restricción de “una tarea por agente”, el problema deja de ser trivial y requiere resolver una asignación global (húngaro / programación lineal entera), no simplemente tomar el máximo de cada columna. **Lección:** las restricciones de exclusión transforman una selección voraz local en un problema de optimización combinatoria global.

13 Negociación y equilibrio de Nash

Competencia asociada: STC0103-B — Generación de modelos computacionales

El análisis mediante el **Equilibrio de Nash** es una competencia explícita del curso: identificar la teoría formal (teoría de juegos) que sustenta el comportamiento estable de agentes que interactúan.

Los agentes también pueden negociar: “yo realizo la tarea 2 si tú realizas la tarea 4”. Para dos agentes, $U_1(a_1, a_2)$ y $U_2(a_1, a_2)$ son las utilidades respectivas. Un **equilibrio de Nash** ocurre cuando ningún agente obtiene beneficio cambiando unilateralmente su estrategia:

$$U_i(a_i^*, a_{-i}^*) \geq U_i(a_i, a_{-i}^*) \quad \forall a_i.$$

Ejercicio de opción múltiple 5

Dos compañeros de casa deciden si *Cooperar* (C) o *No cooperar* (N) en las tareas domésticas.

La matriz de pagos (U_1, U_2) es:

	C_2	N_2
C_1	(3, 3)	(0, 5)
N_1	(5, 0)	(1, 1)

¿Cuál es el equilibrio de Nash en estrategias puras?

- A) (C_1, C_2) , porque es el mejor resultado conjunto.
- B) (N_1, N_2) , con pagos (1, 1).
- C) (C_1, N_2) , porque un jugador coopera.
- D) No existe equilibrio de Nash.

Respuesta correcta y explicacion detallada

La respuesta correcta es la B. Este es el clásico **dilema del prisionero**, y su lección es contraintuitiva.

Método: buscar la mejor respuesta de cada jugador y ver dónde coinciden.

Paso 1. Mejor respuesta del Jugador 1 (fijando lo que hace el 2, comparo mis filas):

- Si el Jugador 2 juega C_2 : comparo $U_1(C_1, C_2) = 3$ vs $U_1(N_1, C_2) = 5$. Como $5 > 3$, **prefiero** N_1 .
- Si el Jugador 2 juega N_2 : comparo $U_1(C_1, N_2) = 0$ vs $U_1(N_1, N_2) = 1$. Como $1 > 0$, **prefiero** N_1 .

Sin importar lo que haga el otro, al Jugador 1 le conviene N_1 : es una *estrategia dominante*.

Paso 2. Mejor respuesta del Jugador 2 (por simetría de la matriz):

- Si el Jugador 1 juega C_1 : $U_2(C_1, C_2) = 3$ vs $U_2(C_1, N_2) = 5 \Rightarrow$ prefiere N_2 .
- Si el Jugador 1 juega N_1 : $U_2(N_1, C_2) = 0$ vs $U_2(N_1, N_2) = 1 \Rightarrow$ prefiere N_2 .

N_2 también es estrategia dominante.

Paso 3. Intersección. Ambos juegan su estrategia dominante: el perfil (N_1, N_2) con pagos (1, 1). Verificamos que es Nash: desde (1, 1), si el Jugador 1 se desviara a C_1 obtendría $0 < 1$ (peor); igual el Jugador 2. Ninguno gana desviándose unilateralmente $\Rightarrow$ **es equilibrio de Nash**.

Por qué las demás son incorrectas:

- **A** confunde *óptimo de Pareto* con *equilibrio de Nash*. $(C_1, C_2) = (3, 3)$ es socialmente mejor que (1, 1), pero *no es estable*: cada jugador tiene incentivo a traicionar y ganar 5. La tragedia del dilema es precisamente que el equilibrio (1, 1) es peor para todos que la cooperación, y aun así es el resultado racional.
- **C** no es equilibrio: en (C_1, N_2) el Jugador 1 gana 0 y se desviaría a N_1 para ganar 1.
- **D** es falsa: acabamos de exhibir uno.

Lección: un equilibrio de Nash es un punto de *estabilidad estratégica*, no necesariamente de eficiencia social. Diseñar mecanismos (reglas, contratos, reputación) que alineen el equilibrio con el óptimo social es un objetivo central del diseño de sistemas multiagentes.

14 Q-Learning y aprendizaje por refuerzo

Competencia asociada: STC0104-B — Implementación de modelos computacionales

Implementar Q-Learning y su versión multiagente ejercita la **implementación de algoritmos de aprendizaje** y su interoperabilidad, eligiendo el más adecuado para el problema dado.

La regla de actualización de Q-Learning es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

donde α es la tasa de aprendizaje, γ el factor de descuento, r la recompensa y $Q(s, a)$ el valor esperado de ejecutar a en s . Valores típicos: $\alpha = 0.1$, $\gamma = 0.95$, $\epsilon = 0.1$, con política ϵ -greedy.

Algoritmo 3: Q-Learning tabular con política ϵ -greedy

Inicializar: $Q(s, a) \leftarrow 0$; parámetros α, γ, ϵ

```

1 mientras no se alcance el criterio de paro hacer
2   observar estado  $s$ ;
3   si  $\text{aleatorio}() < \epsilon$  entonces
4      $a \leftarrow$  acción aleatoria;                                ▷ exploración
5   si no y
6      $a \leftarrow \arg \max_{a'} Q(s, a')$ ;                            ▷ explotación
7   ejecutar  $a$ , observar recompensa  $r$  y nuevo estado  $s'$ ;
8    $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ ;
9    $s \leftarrow s'$ ;
```

Ejercicio de opción múltiple 6

Un agente tiene $Q(s, a) = 2$. Ejecuta a , recibe $r = 10$ y llega a s' donde $\max_{a'} Q(s', a') = 5$. Con $\alpha = 0.1$ y $\gamma = 0.9$, ¿cuál es el nuevo valor de $Q(s, a)$?

- A) $Q(s, a) = 2.0$ (no cambia).
- B) $Q(s, a) = 3.25$.
- C) $Q(s, a) = 14.5$.
- D) $Q(s, a) = 12.5$.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

Paso 1. Identificar cada componente de la regla de actualización.

$$Q_{\text{nuevo}} = Q + \alpha [r + \gamma \max_{a'} Q(s', a') - Q],$$

con $Q = 2$, $\alpha = 0.1$, $r = 10$, $\gamma = 0.9$ y $\max_{a'} Q(s', a') = 5$.

Paso 2. Calcular el “objetivo” (*target*). Es la mejor estimación actual de lo que *debería* valer $Q(s, a)$, combinando la recompensa recibida con el valor futuro descontado:

$$\text{target} = r + \gamma \max_{a'} Q(s', a') = 10 + 0.9 \times 5 = 10 + 4.5 = 14.5.$$

Paso 3. Calcular el error temporal (*TD error*). Es la diferencia entre el objetivo y el valor actual:

$$\delta = \text{target} - Q = 14.5 - 2 = 12.5.$$

Paso 4. Actualizar dando un paso de tamaño α en dirección del error.

$$Q_{\text{nuevo}} = Q + \alpha \delta = 2 + 0.1 \times 12.5 = 2 + 1.25 = \mathbf{3.25}.$$

Por qué las demás son incorrectas (cada una es una confusión típica):

- **A** (2.0) equivale a usar $\alpha = 0$: el agente nunca aprende, ignora toda experiencia nueva.
- **C** (14.5) equivale a usar $\alpha = 1$: el agente reemplaza su valor por el objetivo completo, sin suavizado; esto olvida todo lo aprendido antes y hace el aprendizaje inestable ante el ruido.
- **D** (12.5) confunde el *TD error* δ con el valor final; δ es solo el ingrediente que se multiplica por α , no el resultado.

Interpretación: con $\alpha = 0.1$ el agente se mueve solo un 10% hacia el objetivo en cada paso, lo que estabiliza el aprendizaje. **Lección:** la tasa de aprendizaje α controla cuánto “cree” el agente en cada nueva observación; valores pequeños aprenden despacio pero de forma estable, valores grandes aprenden rápido pero oscilan.

15 Métricas, visualización y proyecto final

Competencia asociada: STC0104-B — Implementación de modelos computacionales

Instrumentar la simulación con métricas y graficarlas con bibliotecas (`matplotlib`, `networkx`) desarrolla el manejo de **bibliotecas y APIs gráficas** mencionado en la competencia.

Durante una simulación conviene almacenar: recompensa instantánea R_t ; recompensa acumulada $G_t = \sum_{\tau=0}^t R_\tau$; tareas completadas $N_{\text{tasks}}(t)$; energía $E_i(t)$; distancia $D_i(t)$; carga de trabajo $W_i(t)$; y conflictos $C(t)$. Dos indicadores derivados útiles son la eficiencia

$$\eta = \frac{\text{tareas completadas}}{\text{tiempo total}},$$

y el balance de carga

$$B = 1 - \frac{\sigma(W)}{\mu(W) + \epsilon},$$

donde $\sigma(W)$ es la desviación estándar de la carga, $\mu(W)$ la carga promedio y ϵ evita la división entre cero. Valores altos de B representan asignaciones más equilibradas.

15.1 Ejemplo de graficación en Python

```
1 import matplotlib.pyplot as plt
2
3 tiempo = [0, 1, 2, 3, 4]
4 recompensa = [2, 4, 5, 9, 12]
5
6 plt.plot(tiempo, recompensa, marker="o")
7 plt.xlabel("Tiempo")
8 plt.ylabel("Recompensa acumulada")
9 plt.title("Desempeno del agente")
10 plt.grid()
11 plt.show()
```

15.2 Arquitectura sugerida para el proyecto final

El proyecto **Multi-Agent Daily Planner** recibe agentes, horarios, ubicaciones, tareas, prioridades, recursos y restricciones; genera un plan; y ante eventos inesperados replanifica. El flujo es:

Observaciones ↓ Estado del sistema ↓ Planificador ↓
Asignación multiagente ↓ Ejecución ↓ Eventos ↓
Replanificación

16 Ejercicios de implementación

Los siguientes ejercicios prácticos desarrollan la competencia **STC0104-B**. Se conservan como retos de implementación abiertos.

1. **Selección de tareas (voraz).** Con cinco tareas de prioridad y duración dadas y $T_{\max} = 6$ h, defina $U_j = \text{prioridad}_j / \text{duración}_j$ y seleccione con estrategia voraz.
2. **Dos agentes, cinco tareas.** Dada una matriz de costos, asigne cada tarea al agente de menor costo con `np.argmin`.
3. **Sistema de subastas.** Tres agentes, cuatro tareas, pujas aleatorias en $[0, 10]$; añada la regla de máximo dos tareas por agente.
4. **Agentes y energía.** Tres robots con energía $[80, 60, 40]$ y tareas de costo $[20, 30, 50]$; un robot acepta si $E_i - C_j > E_{\min} = 20$.
5. **Recompensa multiobjetivo.** $R = 3P - 2T - 4C$ sobre diez tareas aleatorias; calcule y ordene.
6. **Cuadrícula 10×10 .** Dos agentes, tres objetivos, obstáculos; recompensa +10 objetivo, -1 movimiento, -5 colisión. Compare estrategia aleatoria vs voraz.
7. **Q-Learning.** Implemente el Algoritmo 3 con $\alpha = 0.1$, $\gamma = 0.95$, $\epsilon = 0.1$.

8. **Multi-Agent Q-Learning.** Dos agentes con tablas Q_1, Q_2 ; +10 si ambos alcanzan su objetivo, -10 si chocan. Analice si aprenden a evitar colisiones.
9. **Planificación familiar dinámica.** Cuatro agentes {padre, madre, hijo₁, hijo₂} y actividades con duración, prioridad, ubicación y horario. Minimice $J = w_1(\text{tiempo}) + w_2(\text{desplazamiento}) + w_3(\text{tareas incumplidas})$.
10. **Evento inesperado.** Sobre el ejercicio anterior, a mitad de simulación genere un evento (tráfico, cancelación, enfermedad, vehículo no disponible, tarea urgente). Ejecute Detectar → Actualizar → Replanificar y compare plan fijo vs dinámico.

17 Demostración científica del funcionamiento del sistema

Competencia de área: Demostración del funcionamiento de sistemas computacionales

Esta sección desarrolla la competencia del área de Computación y Tecnologías de Información (evaluada en **TI2018**, **TC2008B** y **MR2002B**): *demostrar el funcionamiento de sistemas computacionales mediante evidencias empíricas o teóricas, aplicando metodologías científicas*. No basta con implementar el sistema multiagente: hay que **probar que funciona** seleccionando las reglas fundamentales de su estructura, formulando hipótesis y validando el prototipo con experimentos controlados.

Construir un simulador multiagente no demuestra nada por sí solo. Un sistema “que corre” puede estar produciendo resultados por azar, por un *bug* que se compensa con otro, o bajo condiciones que no generalizan. La competencia exige aplicar el **método científico** al propio sistema computacional.

17.1 El ciclo de validación científica de un prototipo

El siguiente ciclo estructura la demostración empírica del funcionamiento de un sistema multiagente:

1. **Pregunta.** ¿El sistema produce el comportamiento deseado? Ej.: “¿la coordinación por subasta reduce los conflictos frente a la asignación aleatoria?”
2. **Hipótesis falsable.** Enunciado que puede refutarse con datos. Ej.: “ H_1 : la subasta reduce el número medio de conflictos en al menos 30 % respecto de la asignación aleatoria”. La hipótesis nula H_0 afirma que no hay diferencia.
3. **Diseño experimental.** Definir variable **independiente** (el mecanismo de coordinación), variables **dependientes** (conflictos, eficiencia η , balance B), variables **control** (número de agentes, mapa, semilla) y número de réplicas.
4. **Ejecución con réplicas y semillas.** Repetir cada condición con múltiples semillas aleatorias para estimar la variabilidad, no un solo corrida.
5. **Análisis estadístico.** Medias, intervalos de confianza y una prueba de hipótesis. Reportar tamaño del efecto, no solo el valor p .
6. **Conclusión.** Aceptar o rechazar H_0 *con la evidencia*, reconociendo los límites de validez del experimento.

Evidencia teórica vs. empírica. Un sistema puede validarse de dos formas complementarias. La **teórica** demuestra propiedades por argumento formal (p. ej., que Q-Learning tabular converge al Q^* óptimo bajo ciertas condiciones, o que un mecanismo de subasta es *a prueba de estrategias*). La **empírica** las verifica midiendo el prototipo en ejecución. La competencia acepta ambas, pero exige rigor: reglas fundamentales explícitas y metodología reproducible.

Ejercicio de opción múltiple 7

Un equipo afirma: “ejecutamos nuestro planificador multiagente una vez, completó todas las tareas, luego el sistema funciona”. ¿Cuál es la falla metodológica **más grave** desde el punto de vista del método científico?

- A) Ninguna: si completó las tareas, queda demostrado.
- B) Una sola ejecución no controla la aleatoriedad ni permite estimar variabilidad; falta replicación con distintas semillas y una comparación contra una línea base.
- C) El error es no haberlo programado en C++ para que sea más rápido.
- D) El error es no haber usado aprendizaje profundo.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

Demostrar el funcionamiento de un sistema es una afirmación *científica*, y una sola observación casi nunca constituye evidencia suficiente. El problema tiene tres capas:

- 1. Falta de replicación.** Los sistemas multiagentes suelen contener aleatoriedad (posiciones iniciales, política ϵ -greedy, generación de tareas). Una única corrida es una sola muestra de una distribución de resultados: podría haber sido afortunada. Se necesitan *múltiples réplicas con distintas semillas* para estimar la media y su dispersión.
- 2. Ausencia de grupo de comparación (*baseline*).** “Completó todas las tareas” no dice si el mecanismo aporta algo. Quizá una asignación *aleatoria* también las completa. Sin una línea base contra la cual contrastar, no hay forma de atribuir el resultado al diseño del sistema.
- 3. Falta de hipótesis falsable y métrica objetivo.** “Funciona” es vago. La afirmación debe volverse medible: *qué métrica, cuánto mejor, y respecto de qué*.

Por qué las demás son incorrectas:

- **A** confunde “ocurrió una vez” con “se demostró”; es exactamente el error que el método científico previene.
- **C** y **D** apuntan a decisiones de implementación (lenguaje, técnica) que son irrelevantes para la *validez* de la demostración. Un sistema en Python puro validado con rigor demuestra más que uno en C++ con aprendizaje profundo ejecutado una sola vez.

Lección: demostrar el funcionamiento = hipótesis falsable + línea base + replicación + análisis estadístico, no una anecdótica ejecución exitosa.

Ejercicio de opción múltiple 8

Se comparan dos coordinadores. Con 30 réplicas cada uno, los conflictos medios son: *Subasta* $\mu_A = 4.2$ (desv. $\sigma_A = 1.1$); *Aleatorio* $\mu_R = 7.8$ ($\sigma_R = 1.3$). ¿Qué conclusión está **correctamente** sustentada?

- A) No se puede concluir nada porque los rangos $[\mu \pm \sigma]$ podrían tocarse.
- B) La subasta reduce los conflictos; la diferencia de medias (3.6) es grande frente a las desviaciones, y una prueba t sobre 30 réplicas permitiría confirmar significancia y estimar el tamaño del efecto.
- C) La subasta es peor porque $4.2 < 7.8$ significa menos actividad.
- D) Basta reportar que la subasta “se ve mejor” en la gráfica.

Respuesta correcta y explicacion detallada

La respuesta correcta es la B.

Paso 1. Leer las magnitudes correctamente. En “conflictos”, *menos es mejor*. La subasta promedia 4.2 y la aleatoria 7.8: la subasta genera menos conflictos. Esto descarta la opción C, que malinterpreta un menor número de conflictos como algo negativo.

Paso 2. Comparar la diferencia con la variabilidad. La diferencia de medias es

$$\Delta = \mu_R - \mu_A = 7.8 - 4.2 = 3.6.$$

Las desviaciones (1.1 y 1.3) son pequeñas frente a $\Delta = 3.6$: la señal es mucho mayor que el ruido. Como estimación rápida del error estándar de la diferencia:

$$SE = \sqrt{\frac{\sigma_A^2}{n} + \frac{\sigma_R^2}{n}} = \sqrt{\frac{1.1^2}{30} + \frac{1.3^2}{30}} = \sqrt{\frac{1.21 + 1.69}{30}} = \sqrt{0.0967} \approx 0.31.$$

El estadístico aproximado sería $t \approx \Delta/SE = 3.6/0.31 \approx 11.6$, muy por encima de los umbrales usuales: la diferencia es estadísticamente significativa.

Paso 3. Reportar tamaño del efecto. Una d de Cohen, usando una desviación combinada $\sigma_p \approx 1.2$, da

$$d = \frac{\Delta}{\sigma_p} = \frac{3.6}{1.2} = 3.0,$$

un efecto enorme. Reportar el tamaño del efecto (y no solo el valor p) es parte de una demostración científica seria.

Por qué las demás son incorrectas:

- **A** es demasiado conservadora: aunque los intervalos $[\mu \pm \sigma]$ se acercaran, la prueba estadística correcta trabaja con el *error estándar de la media* ($\sigma/\sqrt{n}$), que aquí es pequeñísimo gracias a las 30 réplicas. La conclusión sí es posible.
- **C** invierte el significado de la métrica.
- **D** sustituye el análisis por una impresión visual; una gráfica ilustra, pero no reemplaza la prueba de hipótesis.

Lección: demostrar una mejora exige contrastar la diferencia contra la variabilidad (prueba de hipótesis) y cuantificar cuán grande es (tamaño del efecto), aprovechando las réplicas para reducir el error estándar.

Ejercicio de opción múltiple 9

Al validar un agente de Q-Learning, ¿cuál de las siguientes es la **evidencia empírica más convincente** de que el agente realmente *aprendió* (y no de que tuvo suerte)?

- A) Una gráfica de una sola corrida donde la recompensa sube al final.
- B) La curva de recompensa acumulada promediada sobre varias semillas muestra una tendencia creciente y estable que supera de forma consistente a una política aleatoria de referencia.
- C) El código compila sin errores.
- D) El agente alcanzó la meta en el último episodio.

Respuesta correcta y explicación detallada

La respuesta correcta es la B.

La pregunta clave del método científico aquí es: *¿cómo distingo aprendizaje real de una coincidencia afortunada?* La respuesta es **consistencia a través de la aleatoriedad y superioridad sobre una línea base**.

Por qué la opción B es la correcta:

- *Promediar sobre varias semillas* elimina el efecto de una corrida afortunada: si la tendencia creciente persiste en promedio, es una propiedad del algoritmo, no del azar.
- *Tendencia creciente y estable* es la firma del aprendizaje: el agente mejora su desempeño con la experiencia y luego se estabiliza (converge).
- *Superar consistentemente a una política aleatoria* demuestra que el aprendizaje aporta valor por encima de actuar al azar; sin esa comparación, “subir” no significa nada.

Por qué las demás son insuficientes:

- **A** es una sola muestra: la subida final pudo ser ruido. Una golondrina no hace verano.
- **C** confunde *ausencia de errores de compilación* con *corrección del comportamiento*. Un programa que compila puede aprender mal o no aprender.
- **D** es un único evento puntual; alcanzar la meta una vez es compatible con pura suerte, sobre todo en entornos pequeños.

Lección: la evidencia empírica robusta de aprendizaje es una curva agregada (media $\pm$ dispersión sobre semillas) que domina de forma estable a una referencia, no un evento aislado ni un criterio de ingeniería como “compila”.

17.2 Retos de demostración científica (implementación)

Los siguientes retos desarrollan directamente la competencia del área. Cada uno exige **formular una hipótesis, diseñar el experimento, ejecutarlo con réplicas y concluir con evidencia**.

1. **Subasta vs. aleatorio (evidencia empírica).** Hipótesis: la coordinación por subasta reduce los conflictos frente a la asignación aleatoria. Ejecute ambos coordinadores con ≥ 30 semillas, registre conflictos y eficiencia η , grafique medias con barras de error y aplique una prueba t . Reporte el tamaño del efecto y concluya si se rechaza H_0 .
2. **Convergencia de Q-Learning (evidencia empírica + teórica).** Demuestre empíricamente que el agente converge: grafique la recompensa acumulada promediada sobre ≥ 20 semillas y muestre que se estabiliza. Argumente teóricamente por qué, con ϵ decreciente y visitas suficientes, la tabla Q se aproxima a Q^* .
3. **Estudio de escalabilidad (diseño experimental).** Variable independiente: número de agentes $N \in \{2, 4, 8, 16\}$. Variables dependientes: eficiencia global y número de conflictos. Formule una hipótesis sobre cómo escalan, ejecute con réplicas y presente la curva N vs. métrica. ¿Se confirma la hipótesis?
4. **Ablación de reglas fundamentales.** Identifique tres “reglas fundamentales” de su sistema (p.ej.: la restricción de exclusión de recursos, la replanificación ante eventos, la penalización por varianza). Desactive una a la vez (*ablation study*) y mida el impacto en las métricas. Esto demuestra *cuáles* reglas son responsables del funcionamiento correcto.
5. **Robustez ante eventos (validación de replanificación).** Hipótesis: la planificación dinámica se recupera mejor que el plan fijo ante perturbaciones. Inyecte eventos aleatorios, mida el “tiempo de recuperación” y las tareas incumplidas en ambos esquemas, con réplicas, y contraste estadísticamente.
6. **Reproducibilidad.** Documente semillas, parámetros y versión del código de modo que un tercero reproduzca sus figuras. Entregue un script que regenere todos los resultados. La reproducibilidad es un requisito de la metodología científica, no un extra.

17.3 Plantilla de reporte científico

Para entregar la evidencia de la competencia, estructure un breve reporte con: (1) pregunta e hipótesis; (2) diseño experimental (variables independiente, dependientes y control; número de réplicas); (3) resultados (tablas y figuras con barras de error); (4) análisis estadístico (prueba y tamaño del efecto); (5) conclusión que acepta o rechaza H_0 y discute límites de validez.

```

1 import numpy as np
2 from scipy import stats
3
4 def experimento(coordinador, n_semillas=30):
5     """Ejecuta el sistema con varias semillas y regresa los conflictos.
6         """
7     conflictos = []
8     for semilla in range(n_semillas):
9         np.random.seed(semilla)
10        sistema = construir_sistema(coordinador=coordinador)

```

```

10         resultado = sistema.simular()
11         conflictos.append(resultado["conflictos"])
12     return np.array(conflictos)
13
14     # Recolectar evidencia empirica con replicas
15     c_subasta = experimento("subasta")
16     c_aleatorio = experimento("aleatorio")
17
18     # Estadisticos descriptivos
19     print("Subasta: media =", c_subasta.mean(), "+/-", c_subasta.std())
20     print("Aleatorio:media =", c_aleatorio.mean(), "+/-", c_aleatorio.std())
21
22     # Prueba de hipotesis (t de Student para muestras independientes)
23     t, p = stats.ttest_ind(c_subasta, c_aleatorio)
24
25     # Tamano del efecto (d de Cohen)
26     sp = np.sqrt((c_subasta.var() + c_aleatorio.var()) / 2)
27     d = (c_aleatorio.mean() - c_subasta.mean()) / sp
28
29     print(f"t = {t:.2f}, p = {p:.4g}, d de Cohen = {d:.2f}")
30     print("Se rechaza H0" if p < 0.05 else "No se rechaza H0")

```

18 De algoritmos clásicos a agentes modernos

La evolución conceptual del área puede leerse como una cadena:

Heurística → Búsqueda → Optimización → MDP → RL → MARL → Agentes autónomos modernos.

Un agente basado en un modelo de lenguaje puede incorporar además razonamiento, memoria, planificación, uso de herramientas, comunicación, negociación y replanificación, ejecutando continuamente:

Observe → Reason → Plan → Act → Observe.

19 Evaluación de Sistemas Multiagentes con agentes de IA

Competencia asociada: Integración de competencias — modelar, especificar, implementar y validar

Esta sección propone un esquema de evaluación en el que el estudiante debe **comprender un sistema multiagente, expresarlo con precisión, dirigir a un agente de IA para construirlo y evaluar su comportamiento**. La calificación no depende solamente de que “la aplicación funcione”, sino de la calidad del conocimiento disciplinar, de la especificación, de la iteración y de la validación empírica.

19.1 Motivación del enfoque

El diseño de esta evaluación se inspira en el trabajo de Thorgeirsson, Weidmann y Su sobre *vibe coding* [1]. En ese estudio, el desempeño al programar mediante lenguaje natural se analizó separando tres tipos de capacidades: conocimiento de Ciencias Computacionales, comunicación escrita y razonamiento general. Los autores encontraron que tanto el conocimiento de computación como la habilidad de escritura se asociaban positivamente con el desempeño, y que el conocimiento de

computación aportaba aproximadamente el doble de varianza única que la escritura en el modelo conjunto. También encontraron que la calidad de los prompts estaba fuertemente relacionada con el resultado final.

La consecuencia pedagógica para Sistemas Multiagentes Inteligentes es importante: **permitir el uso de un LLM no elimina la necesidad de saber multiagentes**. Por el contrario, permite evaluar una competencia más rica:

conocimiento de MAS $\rightarrow$ modelo mental $\rightarrow$ especificación $\rightarrow$ interacción con IA $\rightarrow$ sistema ejecutable $\rightarrow$

19.2 Constructos que se medirán

Para evitar que una sola calificación mezcle habilidades diferentes, se registrarán cuatro medidas principales:

K_{MAS} = conocimiento disciplinar de Sistemas Multiagentes,

W = calidad de la especificación escrita,

P = calidad de la secuencia de prompts,

A = desempeño funcional del sistema producido con IA.

Opcionalmente puede agregarse una medida breve de razonamiento general G como covariable, si el objetivo es realizar un estudio educativo comparable metodológicamente con el artículo de referencia. Para el uso normal del examen no es indispensable.

Todas las medidas pueden normalizarse al intervalo $[0, 1]$:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}.$$

Esto permite comparar cohortes, tareas y semestres aun cuando las escalas originales sean distintas.

19.3 Principio de diseño: observar antes de especificar

En lugar de entregar al alumno una lista completa de requisitos que pueda copiar directamente al LLM, se le presenta primero un **sistema cotidiano observable**. El alumno debe inferir sus reglas, restricciones, objetivos e interacciones. De esta forma, el examen mide la capacidad de traducir comportamiento observado a una especificación multiagente.

Un escenario base puede ser la organización de una reunión entre amigos:

- cada persona es un agente con horario, ubicación, preferencias y restricciones;
- existen recursos compartidos, por ejemplo automóviles, lugares y presupuesto;
- los agentes pueden proponer horarios, aceptar, rechazar o negociar;
- puede aparecer tráfico, cancelaciones o cambios de disponibilidad;
- el sistema debe encontrar una solución factible y razonablemente justa.

Otros escenarios cotidianos equivalentes son: una familia coordinando actividades, vehículos buscando estacionamiento, robots domésticos compartiendo recursos, alumnos organizando equipos y espacios, repartidores coordinando entregas o agentes digitales administrando citas y pendientes.

19.4 Tres tipos de tareas

Para observar diferentes formas de competencia se proponen tres tareas, siguiendo la lógica de replicación, extensión y descontextualización:

1. **Tarea de replicación.** El alumno observa un sistema funcionando y debe recrear su comportamiento con ayuda de un agente de IA.
2. **Tarea de extensión.** Sobre el sistema anterior se introduce una nueva regla: negociación, falla de un agente, recurso compartido, prioridad, penalización por inequidad o evento inesperado.
3. **Tarea descontextualizada.** El alumno observa comportamiento abstracto sin etiquetas semánticas evidentes. Debe inferir si existe subasta, exclusión mutua, asignación centralizada, cooperación, competencia, consenso u otro mecanismo.

Esta estructura evita que el nombre del problema revele demasiado al modelo y obliga al estudiante a construir la representación conceptual.

20 Examen propuesto: Multiagentes en la vida cotidiana

20.1 Condiciones generales

- Duración sugerida: **90 minutos**.
- Se permite utilizar un agente de IA solamente en las fases indicadas.
- Durante la primera fase no se permite consultar al LLM.
- En la fase de construcción puede usarse ChatGPT, Codex, Claude u otro agente autorizado por el profesor.
- Debe entregarse la **secuencia completa de prompts**; no solamente el resultado final.
- El profesor puede ocultar el código durante una primera ronda de implementación para evaluar la capacidad de especificar mediante lenguaje natural y observación del comportamiento.

20.2 Fase 1 — Comprensión disciplinar sin IA: K_{MAS} (25 puntos)

El alumno observa durante cinco minutos una simulación de coordinación cotidiana. A partir de ella debe identificar:

1. los agentes y el entorno;
2. observaciones locales y estado global;
3. acciones disponibles;
4. objetivos individuales y globales;
5. restricciones y recursos compartidos;
6. mecanismo de coordinación probable;
7. riesgos de conflicto, deadlock, competencia o inequidad.

Además, debe representar formalmente un agente i mediante:

$$A_i = (S_i, O_i, U_i, \pi_i, R_i),$$

donde S_i es su estado interno, O_i sus observaciones, U_i sus acciones, π_i su política y R_i su función de recompensa o utilidad.

20.3 Fase 2 — Especificación escrita: W (20 puntos)

Sin utilizar todavía el agente de IA, el alumno redacta una especificación de 250–400 palabras destinada a un programador que no ha visto la simulación. Debe describir el comportamiento esperado con suficiente precisión para que pueda ser implementado.

La especificación debe incluir obligatoriamente:

- propósito del sistema;
- tipos de agentes;
- variables de estado;
- acciones disponibles;
- reglas de interacción;
- restricciones;
- eventos inesperados;
- criterio observable de éxito.

20.4 Fase 3 — Construcción con agente de IA: P y A (35 puntos)

El alumno dispone de 30 minutos para convertir su especificación en un prototipo ejecutable utilizando un agente de IA. Debe conservar todos sus prompts:

$$\mathcal{P} = \{p_1, p_2, \dots, p_n\}.$$

No se premia escribir un prompt muy largo. Se premia la capacidad de **observar, diagnosticar, corregir y refinar**. Una secuencia esperada puede seguir el ciclo:

especificar $\rightarrow$ ejecutar $\rightarrow$ observar $\rightarrow$ diagnosticar $\rightarrow$ refinar.

El profesor puede restringir durante los primeros 15 minutos la edición manual del código. En ese intervalo todas las modificaciones deben pedirse al agente mediante lenguaje natural. Esto reduce la confusión entre “saber programar directamente” y “saber dirigir un agente programador”.

20.5 Fase 4 — Diagnóstico, robustez y reflexión: D (20 puntos)

Se introduce una perturbación no anunciada. Por ejemplo:

Al aumentar el sistema de 5 a 50 agentes aparecen conflictos: varios agentes seleccionan simultáneamente el mismo recurso y algunos quedan esperando indefinidamente. Explique una causa probable, proponga un mecanismo de coordinación para resolverla y escriba el siguiente prompt que enviaría al agente de IA.

El alumno debe responder con lenguaje técnico y justificar la solución utilizando conceptos del curso: exclusión mutua, negociación, subasta, prioridades, consenso, asignación de tareas, utilidad, comunicación, replanificación o aprendizaje, según corresponda.

21 Rúbrica analítica de evaluación

La calificación total es:

$$E = 0.25K_{MAS} + 0.20W + 0.15P + 0.20A + 0.20D$$

cuando cada componente se expresa en escala de 0 a 100. De manera equivalente, los puntos del examen suman 100.

21.1 Rúbrica global de 100 puntos

Dimensión	Evidencia esperada	Puntos
Conocimiento MAS K_{MAS}	Identifica correctamente agentes, estados, observaciones, acciones, objetivos, restricciones y mecanismo de coordinación; usa formalización coherente con teoría de juegos, MDP/Dec-POMDP, asignación o coordinación cuando corresponda.	25
Especificación escrita W	Expresa con claridad el propósito, descompone el problema, define estados, reglas, restricciones y criterios verificables; lenguaje técnico preciso y sin ambigüedades relevantes.	20
Calidad de prompts P	Los prompts son coherentes, progresivos, verificables y orientados a corregir causas; incluyen contexto suficiente y evitan reescrituras vagas que no agregan información.	15
Desempeño funcional A	El prototipo implementa las reglas observadas, coordina agentes, respeta recursos y restricciones, responde a eventos y produce comportamiento verificable.	20
Diagnóstico y robustez D	Explica fallas con conceptos de MAS, propone una mejora pertinente, anticipa consecuencias y define cómo validar que la modificación resolvió el problema.	20
Total		100

21.2 Rúbrica específica para la escritura y especificación (20 puntos)

Cada criterio se califica de 0 a 4.

Criterio	Descripción del nivel máximo (4 puntos)
Claridad del objetivo	El sistema y su propósito pueden comprenderse sin conocimiento implícito del contexto observado.
Descomposición	Se distinguen agentes, subtareas, estados, acciones, recursos y eventos de manera organizada.
Interacciones	Se especifica quién se comunica o coordina con quién, qué información intercambia y bajo qué reglas.
Variables y restricciones	Las variables relevantes, límites, precedencias, exclusiones y condiciones de transición están expresadas sin ambigüedad.
Verificabilidad	La especificación contiene resultados observables que permiten decidir objetivamente si el comportamiento fue implementado.

Para cada criterio puede utilizarse la siguiente escala:

Puntos	Descriptor
4	Excelente: preciso, completo, coherente y directamente implementable.
3	Adecuado: correcto en lo esencial, con omisiones menores.
2	Parcial: contiene la idea, pero presenta ambigüedades o faltan elementos importantes.
1	Insuficiente: descripción vaga, desorganizada o técnicamente débil.
0	Ausente o incorrecto.

21.3 Rúbrica específica para la calidad de prompts (15 puntos)

Cada criterio se califica de 0 a 3.

Criterio	Descripción del nivel máximo (3 puntos)
Coherencia	La secuencia mantiene el objetivo y hace referencia correcta al estado actual del sistema.
Claridad instruccional	Cada prompt expresa una acción concreta, verificable y sin contradicciones.
Complejidad apropiada	El alumno divide los cambios cuando es conveniente y evita pedir simultáneamente demasiadas modificaciones no relacionadas.
Diagnóstico	Las correcciones se basan en evidencia observada y atacan una causa probable, no solamente el síntoma.
Iteración	La secuencia muestra refinamiento efectivo: prueba resultados, conserva lo que funciona y modifica lo necesario.

21.4 Rúbrica específica del sistema producido (20 puntos)

Cada criterio se califica con 0–4 puntos usando cuatro niveles operativos: *no aceptable, por debajo de lo esperado, cumple lo esperado y excede lo esperado*. Para conservar una escala de cinco valores puede asignarse 0, 1–2, 3 y 4 respectivamente.

Criterio	Evidencia funcional
Comportamiento individual	Cada agente observa, decide y actúa de acuerdo con las reglas inferidas.
Coordinación	El sistema resuelve o evita conflictos y asigna correctamente recursos o tareas.
Objetivo global	El comportamiento conjunto produce el resultado cotidiano solicitado.
Robustez	El sistema responde razonablemente ante cambios, fallas o eventos inesperados.
Escalabilidad básica	El comportamiento no se degrada de forma catastrófica al aumentar el número de agentes en la prueba establecida.

22 Cómo analizar los resultados del grupo

La evaluación puede utilizarse no solamente para asignar una calificación, sino para investigar qué capacidades predicen el buen desempeño con agentes de IA.

22.1 Correlaciones principales

Para cada alumno se registran K_{MAS} , W , P , A y D . La primera pregunta es si el conocimiento disciplinar y la escritura se relacionan con el desempeño funcional:

$$r(K_{MAS}, A), \quad r(W, A).$$

Una hipótesis pedagógica razonable es:

$$H_1 : r(K_{MAS}, A) > 0, \quad H_2 : r(W, A) > 0.$$

22.2 Valor predictivo relativo

Puede ajustarse el modelo:

$$A = \beta_0 + \beta_1 K_{MAS} + \beta_2 W + \epsilon.$$

Para comparar la varianza única explicada por cada predictor se usan regresiones jerárquicas:

$$\Delta R^2_{W|K_{MAS}} \quad \text{y} \quad \Delta R^2_{K_{MAS}|W}.$$

Esto responde una pregunta educativa muy concreta: *¿el alumno logra mejores sistemas principalmente porque domina la teoría de multiagentes, porque sabe especificar bien, o porque ambas capacidades aportan información distinta?*

22.3 Calidad del prompt como mecanismo intermedio

También puede explorarse si la calidad de la escritura se relaciona con el resultado a través de la calidad de los prompts:

$$W \longrightarrow P \longrightarrow A.$$

El modelo de mediación puede escribirse como:

$$\begin{aligned} P &= aW + e_1, \\ A &= c'W + bP + e_2. \end{aligned}$$

El efecto indirecto es:

$$\text{Efecto indirecto} = a b.$$

En un curso normal este análisis debe considerarse **exploratorio**: una correlación o mediación observacional no demuestra causalidad. Sin embargo, permite detectar patrones valiosos para rediseñar instrucciones, tareas y apoyos.

22.4 Comparación por tipo de tarea

Al tener tres tareas diferentes pueden analizarse por separado:

$$\begin{aligned} A_{rep} &= \text{replicación}, \\ A_{ext} &= \text{extensión}, \\ A_{des} &= \text{descontextualizada}. \end{aligned}$$

Esto permite preguntar si la escritura importa más cuando el contexto semántico es pobre, o si el conocimiento de MAS importa más cuando aparecen coordinación, conflictos o restricciones nuevas.

23 Instrumentación y evidencias a conservar

Para que la evaluación sea reproducible deben guardarse, para cada alumno:

1. identificador anónimo del participante;
2. respuesta de conocimiento disciplinar;
3. especificación escrita previa al uso de IA;
4. secuencia completa de prompts con sello de tiempo;
5. versiones o estados intermedios del prototipo;
6. resultado final;
7. puntajes desglosados de la rúbrica;
8. tiempo utilizado en cada fase;
9. resultados de pruebas funcionales automáticas, cuando existan.

Una tabla mínima de datos puede tener la forma:

Alumno	K_{MAS}	W	P	A	D
1	0.82	0.74	0.80	0.86	0.75
2	0.61	0.90	0.88	0.79	0.67
3	0.93	0.58	0.62	0.84	0.92

Esta separación es especialmente importante porque dos estudiantes pueden alcanzar el mismo desempeño funcional A mediante procesos muy distintos. Uno puede apoyarse en un modelo mental sólido de multiagentes; otro puede compensar con una especificación escrita extraordinariamente clara. El objetivo del instrumento es hacer visibles esas diferencias.

24 Recomendaciones para validez y equidad

- Utilizar el mismo modelo de IA, versión, límites y tiempo para todos los alumnos cuando se desee comparar resultados.
- No calificar estilo visual salvo que sea parte explícita del objetivo; priorizar comportamiento observable.
- Definir antes del examen los casos de prueba y la rúbrica funcional.
- Separar la calificación de escritura de la calificación disciplinar para no castigar dos veces el mismo error.
- Realizar doble calificación o calibración entre evaluadores en una muestra de entregas cuando la rúbrica incluya juicio humano.
- Registrar cuál agente de IA y qué versión se utilizó, porque el desempeño del modelo puede cambiar con el tiempo.
- Evitar interpretar correlaciones como causas. El examen permite estudiar asociaciones entre habilidades y desempeño, no demostrar que una habilidad cause directamente la otra.

Resultado pedagógico deseado: el alumno no debe aprender que “usar IA sustituye saber multiagentes”, sino que una buena interacción con un agente de IA exige un modelo mental correcto, una especificación verificable, capacidad de diagnóstico y criterios de validación. El agente genera; el estudiante debe saber *qué pedir, por qué, cómo comprobarlo y qué hacer cuando falla*.

25 Conclusión

Una agenda cotidiana puede parecer sencilla, pero desde una perspectiva computacional involucra simultáneamente:

Scheduling + Routing + Optimization + Constraints + Game Theory + Uncertainty + Learning

Cuando existen múltiples personas, robots o agentes de IA, aparece un problema todavía más interesante:

Multi-Agent Decision Making under Uncertainty

La pregunta *¿qué debería hacer cada agente a continuación?* puede convertirse en una de las más difíciles computacionalmente. En esencia, un agente inteligente moderno ejecuta:

Observar → Decidir → Actuar → Aprender → Replanificar

y un sistema multiagente debe hacer todo esto mientras se coordina con otros agentes que también observan, deciden, actúan y aprenden. Por eso los problemas de la vida cotidiana son excelentes escenarios para estudiar, de forma integrada (**SEG0501-B**), modelar (**STC0103-B**), implementar (**STC0104-B**) y **demostrar científicamente** (**Área CyTI: TI2018, TC2008B, MR2002B**) el funcionamiento de Sistemas Multiagentes Inteligentes.

Referencia metodológica

Referencias

- [1] S. Thorgeirsson, T. B. Weidmann y Z. Su, *Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency*, Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26), Barcelona, 2026.